Demostracion_practica

🚀 Demostración Práctica Interactiva: El Desastre del Mars Pathfinder en FreeRTOS

Duración Estimada: 15 Minutos
Enfoque: Inversión de Prioridades, Herencia de Prioridad y Planificación Preventiva
Plataforma: Simulación QEMU (ARM Cortex-M4) sobre Ubuntu / WSL2 (100% sin hardware físico)

📋 1. Ficha Técnica y Planteamiento del Problema

🎯 Objetivo Pedagógico

Comprender el comportamiento determinista del planificador de **FreeRTOS** observando mediante la terminal cómo se genera una **Inversión de Prioridad** (*Priority Inversion*) al usar un **Semáforo Binario**, y cómo el uso de un **Mutex** resuelve este cuello de botella aplicando el protocolo de **Herencia de Prioridad** (*Priority Inheritance*).

🛰️ El Caso Histórico: Mars Pathfinder (1997)

En la misión a Marte de 1997, la nave espacial *Mars Pathfinder* experimentaba reinicios periódicos del sistema. El motivo: una tarea de alta prioridad (recolección de información de bus de datos) quedaba bloqueada por una tarea de baja prioridad que tenía tomado un recurso compartido. Una tarea de prioridad intermedia (procesamiento de información climatológica) no dejaba ejecutar a la tarea baja para que liberara el recurso.

🛠️ 2. Preparación Inicial del Entorno

Asegúrese de estar en la raíz de su espacio de trabajo en la terminal de Ubuntu / WSL2 y cree la estructura básica si no existe:

```
# Crear estructura de trabajo e ingresar al directorio
mkdir -p ~/Taller-RTOS/src ~/Taller-RTOS/build ~/Taller-RTOS/config
cd ~/Taller-RTOS
```

🚦 Paso 1: Recreando la Falla — Inversión de Prioridades (Semáforo Binario)

En este primer paso, configuramos las tres tareas utilizando un **Semáforo Binario** (`xSemaphoreCreateBinary()`) como mecanismo de sincronización sobre un recurso compartido.

💻 Comando 1: Crear el código fuente inicial (`src/main.cpp`)

Ejecute el siguiente comando en la terminal para escribir el código con la **Opción A (Semáforo Binario)** activada:

```
cat << 'EOF' > src/main.cpp
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include <stdio.h>

SemaphoreHandle_t xRecursoCompartido;
```

```
        // Tarea de Baja Prioridad (Prioridad 1)
        void vTareaBaja(void *pvParameters) {
            for (;;) {
                if (xSemaphoreTake(xRecursoCompartido, portMAX_DELAY) == pdTRUE) {
                    printf("[BAJA]  Obtuvo el recurso compartido (Semáforo).
        ");

                    // Simula procesamiento prolongado con el recurso retenido
                    vTaskDelay(pdMS_TO_TICKS(100));

                    printf("[BAJA]  Liberando el recurso compartido.
        ");
                    xSemaphoreGive(xRecursoCompartido);
                }
                vTaskDelay(pdMS_TO_TICKS(500));
            }
        }

        // Tarea de Media Prioridad (Prioridad 2 - No usa el recurso)
        void vTareaMedia(void *pvParameters) {
            vTaskDelay(pdMS_TO_TICKS(10)); // Breve retardo para permitir que TareaBaja inicie primero
            for (;;) {
                printf("[MEDIA] Tarea Media ejecutando procesamiento intensivo en CPU...
        ");
                vTaskDelay(pdMS_TO_TICKS(50));
            }
        }

        // Tarea de Alta Prioridad (Prioridad 3 - Tarea Crítica)
        void vTareaAlta(void *pvParameters) {
            vTaskDelay(pdMS_TO_TICKS(20)); // Se activa cuando TareaBaja ya posee el recurso
            for (;;) {
                printf("[ALTA]  ¡ALERTA! Tarea Alta solicita el recurso...
        ");
                if (xSemaphoreTake(xRecursoCompartido, portMAX_DELAY) == pdTRUE) {
                    printf("[ALTA]  ¡Recurso obtenido! Atendiendo evento crítico a tiempo.
        ");
                    xSemaphoreGive(xRecursoCompartido);
                }
                vTaskDelay(pdMS_TO_TICKS(400));
            }
        }

        int main(void) {
            // OPCIÓN A: Semáforo Binario (Sin herencia de prioridad)
            xRecursoCompartido = xSemaphoreCreateBinary();
            xSemaphoreGive(xRecursoCompartido);

            if (xRecursoCompartido != NULL) {
                xTaskCreate(vTareaBaja,  "TareaBaja",  1024, NULL, 1, NULL);
                xTaskCreate(vTareaMedia, "TareaMedia", 1024, NULL, 2, NULL);
                xTaskCreate(vTareaAlta,  "TareaAlta",  1024, NULL, 3, NULL);

                vTaskStartScheduler();
            }
            while (1);
        }
        EOF
```

🔨 Comando 2: Compilar y ejecutar en el emulador QEMU

```
        cd ~/Taller-RTOS/build
        cmake -G Ninja -DCMAKE_BUILD_TYPE=Release ..
        ninja
        qemu-system-arm -M netduinoplus2 -cpu cortex-m4 -nographic -kernel stm32h7_edgeai.elf
```

🤖 Salida Observada en la Terminal (Semáforo Binario)

```
[BAJA]  Obtuvo el recurso compartido (Semáforo).
[ALTA]  ¡ALERTA! Tarea Alta solicita el recurso...
[MEDIA] Tarea Media ejecutando procesamiento intensivo en CPU...
[MEDIA] Tarea Media ejecutando procesamiento intensivo en CPU...
[MEDIA] Tarea Media ejecutando procesamiento intensivo en CPU...
[BAJA]  Liberando el recurso compartido.
[ALTA]  ¡Recurso obtenido! Atendiendo evento crítico a tiempo.
```

🔍 Análisis Didáctico: ¿Qué demuestra esta salida sobre FreeRTOS?

1. **Pérdida del Determinismo Temporal**: La `TareaAlta` (Prioridad 3) debería tener el control prioritario de la CPU. Sin embargo, al solicitar el semáforo en uso por `TareaBaja` (Prioridad 1), entra en estado *Blocked*.
2. **Inversión de Prioridad**: Cuando `TareaMedia` (Prioridad 2) despierta, desplaza a `TareaBaja` porque `Prioridad 2 > Prioridad 1`. `TareaBaja` no puede continuar ejecutándose y, por lo tanto, no puede liberar el semáforo.
3. **Efecto Cascada**: La tarea de prioridad media congela indirectamente a la tarea de más alta prioridad del sistema, que ni siquiera requiere interacción con la tarea media.

🛡 Paso 2: Aplicando la Solución — Herencia de Prioridades (Mutex)

Modificamos el código reemplazando el Semáforo Binario por un **Mutex** (`xSemaphoreCreateMutex()`).

💻 Comando 3: Modificar `src/main.cpp` para usar Mutex

Ejecute la actualización directa en la terminal para activar la **Opción B**:

```
cat << 'EOF' > ~/Taller-RTOS/src/main.cpp
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include <stdio.h>

SemaphoreHandle_t xRecursoCompartido;

void vTareaBaja(void *pvParameters) {
    for (;;) {
        if (xSemaphoreTake(xRecursoCompartido, portMAX_DELAY) == pdTRUE) {
            printf("[BAJA]  Obtuvo el recurso compartido (Mutex).
");

            vTaskDelay(pdMS_TO_TICKS(100));

            printf("[BAJA]  Liberando el recurso compartido.
");
            xSemaphoreGive(xRecursoCompartido);
        }
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}

void vTareaMedia(void *pvParameters) {
    vTaskDelay(pdMS_TO_TICKS(10));
    for (;;) {
        printf("[MEDIA] Tarea Media ejecutando procesamiento intensivo en CPU...
");
        vTaskDelay(pdMS_TO_TICKS(50));
    }
}
```

```
        void vTareaAlta(void *pvParameters) {
            vTaskDelay(pdMS_TO_TICKS(20));
            for (;;) {
                printf("[ALTA]  ¡ALERTA! Tarea Alta solicita el recurso...
        ");
                if (xSemaphoreTake(xRecursoCompartido, portMAX_DELAY) == pdTRUE) {
                    printf("[ALTA]  ¡Recurso obtenido de inmediato! Atendiendo evento crítico.
        ");
                    xSemaphoreGive(xRecursoCompartido);
                }
                vTaskDelay(pdMS_TO_TICKS(400));
            }
        }

        int main(void) {
            // OPCIÓN B: Mutex con Herencia de Prioridades activa
            xRecursoCompartido = xSemaphoreCreateMutex();

            if (xRecursoCompartido != NULL) {
                xTaskCreate(vTareaBaja,  "TareaBaja",  1024, NULL, 1, NULL);
                xTaskCreate(vTareaMedia, "TareaMedia", 1024, NULL, 2, NULL);
                xTaskCreate(vTareaAlta,  "TareaAlta",  1024, NULL, 3, NULL);

                vTaskStartScheduler();
            }
            while (1);
        }
        EOF
```

🔨 Comando 4: Recompilar y simular en QEMU

```
        cd ~/Taller-RTOS/build
        ninja
        qemu-system-arm -M netduinoplus2 -cpu cortex-m4 -nographic -kernel stm32h7_edgeai.elf
```

🤖 Salida Observada en la Terminal (Mutex con Herencia de Prioridad)

```
[BAJA]  Obtuvo el recurso compartido (Mutex).
[ALTA]  ¡ALERTA! Tarea Alta solicita el recurso...
[BAJA]  Liberando el recurso compartido.
[ALTA]  ¡Recurso obtenido de inmediato! Atendiendo evento crítico.
[MEDIA] Tarea Media ejecutando procesamiento intensivo en CPU...
```

🔍 Análisis Didáctico: ¿Qué demuestra esta salida sobre FreeRTOS?

1. **Activación de Herencia de Prioridades (*Priority Inheritance*)**: En el instante exacto en que `TareaAlta` (Prioridad 3) solicita el Mutex retenido por `TareaBaja` (Prioridad 1), el Kernel de FreeRTOS eleva temporalmente la prioridad efectiva de `TareaBaja` a **Prioridad 3**.
2. **Bloqueo de la Tarea Intermedia**: Debido a que `TareaBaja` ahora actúa temporalmente con Prioridad 3, la `TareaMedia` (Prioridad 2) ya **no puede desplazarla de la CPU**.
3. **Restauración y Respuesta Inmediata**: `TareaBaja` completa rápidamente su trabajo con el recurso, lo libera, e inmediatamente el Kernel le devuelve su Prioridad 1 original y le entrega el Mutex a `TareaAlta`, garantizando el cumplimiento de plazos estrictos de tiempo real.

⚡ Paso 3: Experimento Avanzado — Cambio Dinámico de Prioridades (`vTaskPrioritySet`)

En este experimento adicional, observaremos cómo el planificador preventivo (*Preemptive Scheduler*) de FreeRTOS reacciona instantáneamente cuando modificamos dinámicamente la prioridad de una tarea en tiempo de ejecución.

💻 Comando 5: Modificar `src/main.cpp` con ajuste dinámico de prioridad

```
cat << 'EOF' > ~/Taller-RTOS/src/main.cpp
#include "FreeRTOS.h"
#include "task.h"
#include <stdio.h>

TaskHandle_t xTaskBajaHandle = NULL;

void vTareaBaja(void *pvParameters) {
    for (;;) {
        printf("[BAJA]  Ejecutando con Prioridad Básica (1)...
");
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void vTareaControladora(void *pvParameters) {
    vTaskDelay(pdMS_TO_TICKS(50));
    printf("
[CONTROL] Elevando dinámicamente la prioridad de TareaBaja a 4...
");

    // Cambiar prioridad de TareaBaja de 1 a 4
    vTaskPrioritySet(xTaskBajaHandle, 4);

    printf("[CONTROL] Prioridad cambiada con éxito.

");
    vTaskDelay(pdMS_TO_TICKS(200));
}

int main(void) {
    xTaskCreate(vTareaBaja, "TareaBaja", 1024, NULL, 1, &xTaskBajaHandle);
    xTaskCreate(vTareaControladora, "Control", 1024, NULL, 2, NULL);

    vTaskStartScheduler();
    while (1);
}
EOF
```

🔨 Comando 6: Recompilar y ejecutar

```
cd ~/Taller-RTOS/build
ninja
qemu-system-arm -M netduinoplus2 -cpu cortex-m4 -nographic -kernel stm32h7_edgeai.elf
```

🤖 Salida Observada en la Terminal

```
[BAJA]  Ejecutando con Prioridad Básica (1)...

[CONTROL] Elevando dinámicamente la prioridad de TareaBaja a 4...
[BAJA]  Ejecutando con Prioridad Básica (1)...
[CONTROL] Prioridad cambiada con éxito.
```

🔍 Análisis Didáctico: ¿Qué demuestra esta salida sobre FreeRTOS?

1. **Conmutación Inmediata por Excepción `PendSV`**: Al llamar a `vTaskPrioritySet(xTaskBajaHandle, 4)`, el Kernel

evalúa si la tarea modificada tiene ahora mayor prioridad que la tarea actual (`Control`, Prioridad 2).
2. **Comportamiento Preventivo**: Al convertirse `TareaBaja` en la tarea de mayor prioridad lista para ejecutar (Prioridad 4), el Kernel interrumpe de inmediato a la tarea `Control` y conmuta el contexto hacia `TareaBaja` antes de permitir que `Control` continúe la impresión de su siguiente mensaje.

📊 4. Resumen Comparativo de Mecanismos en FreeRTOS

Característica	Semáforo Binario (`xSemaphoreCreateBinary`)	Mutex (`xSemaphoreCreateMutex`)
Uso Principal	Sincronización de eventos (ISRs a Tareas)	Exclusión mutua de recursos compartidos
Herencia de Prioridad	✖ No disponible	✔ Habilitada automáticamente
Propietario (*Ownership*)	Ninguno (Cualquier tarea/ISR puede darlo)	Poseído por la tarea que lo tomó
Uso en ISRs	✔ `xSemaphoreGiveFromISR()`	✖ Prohibido dentro de ISRs
Prevención de Inversión	✖ Sufre de Inversión de Prioridad	✔ Previene la Inversión de Prioridad